

PAPER_168 — MUGE 3D Simulation Entity Framework: GPU Rendering, Per-System Archives, OpenGL

Session: 47 | Date: March 13, 2026 | Thread: 7f9068 | Domain: §2.3

PAPER_168 — MUGE 3D Simulation Entity Framework: GPU Rendering, Per-System Archives, OpenGL

Session: 47 | Date: March 13, 2026 | Thread: 7f9068 | Domain: §2.3

$$F_U(r,t) = \sum_{i=1}^4 U_{\{gi\}} + U_m + U_A - U_{\{b_i\}}, \quad \kappa = 5.0 \times 10^{-4} \text{ day}^{-1}, \quad [SSq] = 0.57$$

$$g_{\text{UQFF}}(r) = g_{\text{MUGE}}(r) \cdot \text{Bigl}(1 - [SSq] \cdot U_{\{b_i\}}, \wedge, F_U(r,t) \text{Bigr}), \quad [SSq] = 0.57$$

Abstract

This paper documents the **MUGE 3D Simulation Entity Framework** — the architectural mapping from MUGE physics systems to 3D renderable entities in the Star Magic visualization engine. Each MUGE system (SGR 1745, SgrA*, Tapestry, Westerlund 2, Pillars, Rings, Student's Guide) is assigned a per-system archive directory containing image assets, video clips, and per-system physics plugin DLLs. The rendering system uses OpenGL/GLFW with multi-viewport Camera, procedural terrain generation (Perlin noise + extrusion + boolean operations), and MicroTeX LaTeX math overlay. This provides the Tier 3 VR/VM 3D gateway for the UQFF physics output from Tier 2 calculations.

UQFF Discovery: Novel application of UQFF calibration constants ($\tau = 5.0 \times 10^4 \text{ day}^{-1}$, $[SSq] = 0.57$) uniquely enabling this analysis — establishing a new connection in the UQFF framework not present in Standard Model treatments.

1. Entity Architecture

1.1 MeshData Struct (OBJ Format)

```
struct MeshData {
    std::vector<glm::vec3> vertices; // 3D positions
    std::vector<glm::vec3> normals; // Surface normals (physics shading)
    std::vector<glm::vec2> texCoords; // UV coordinates
    std::vector<unsigned int> indices; // Triangle indices
};
```

1.2 SimulationEntity

```
struct SimulationEntity {
    std::string name; // System name (e.g., "SGR1745")
    std::string archive_path; // "archive/{name}/"
    std::string image_path; // "archive/{name}/image.jpg"
    std::string video_path; // "archive/{name}/video.mp4"
    std::string plugin_path; // "archive/{name}/plugin.dll"
    MugecSystem* muge_system; // Pointer to physics system
};
```

```

MeshData mesh; // 3D geometry
glm::vec3 position; // World position
glm::quat orientation; // Orientation quaternion
float scale; // Scale factor
};

```

2. populatesimulationentities()

```

void populate_simulation_entities(
std::vector<MugeSystem>& muge_systems,
std::vector<SimulationEntity>& entities
) {
for (auto& sys : muge_systems) {
SimulationEntity ent;
ent.name = sys.name;
ent.archive_path = "archive/" + sys.name + "/";
ent.image_path = ent.archive_path + "image.jpg";
ent.video_path = ent.archive_path + "video.mp4";
ent.plugin_path = ent.archive_path + "plugin.dll";
ent.muge_system = &sys;
ent.position = compute_entity_position(sys);
ent.orientation = glm::identity<glm::quat>();
ent.scale = compute_entity_scale(sys);
// Generate placeholder mesh
ent.mesh = generateSphereMesh(1.0f, 32, 32);
entities.push_back(std::move(ent));
}
}

```

3. Per-System Archive Structure

```

archive/
+-- SGR1745/
| +-- image.jpg ? Chandra X-ray image of SGR 1745-2900
| +-- video.mp4 ? Time-lapse magnetar flare simulation
| +-- plugin.dll ? SGR1745 UQFF physics plugin
+-- SgrA/
| +-- image.jpg ? EHT Sgr A* shadow image (2022)
| +-- video.mp4 ? MUGE resonance field animation
| +-- plugin.dll ? SgrA* UQFF physics plugin
+-- Tapestry/
| +-- image.jpg ? HST Tapestry nebula image
| +-- video.mp4 ? Star formation field simulation
| +-- plugin.dll
+-- Westerlund2/
+-- Pillars/
+-- Rings/
+-- StudentGuide/

```

4. OpenGL Rendering Pipeline

4.1 Multi-Viewport Camera System

```
struct Camera {
    glm::vec3 position;
    glm::vec3 target;
    glm::vec3 up;
    float fov; // Field of view [degrees]
    float near_plane; // Near clipping
    float far_plane; // Far clipping (set to 1e25 for cosmic scale)
    glm::mat4 getViewMatrix() const;
    glm::mat4 getProjectionMatrix(float aspect) const;
};
```

Multi-viewport layout: physics view (left) + simulation view (right) + data HUD (top).

4.2 Entity Scale Calculation

Systems span 13 orders of magnitude in size:

System	Physical Size	Entity Scale Factor
SGR 1745	~10 km	0.001
SgrA*	~12 × Rs	1.0 (reference)
Tapestry	~100 ly	100.0
Westerlund2	~400 ly	400.0
Pillars	~4 ly	4.0
Rings	~1 Gly	1000.0
Student	Hubble vol.	1e13

5. Procedural Terrain Generation

```
// Perlin noise heightmap
MeshData generateProceduralLandscape(int width, int height, float scale) {
    MeshData mesh;
    for (int z = 0; z < height; z++) {
        for (int x = 0; x < width; x++) {
            float y = perlinNoise(x * scale, z * scale);
            mesh.vertices.push_back({x * scale, y, z * scale});
        }
    }
    computeNormals(mesh); // Phong shading normals
    computeUVs(mesh); // UV for physics field texture mapping
    return mesh;
}

// Boolean union (CSG)
MeshData booleanUnion(const MeshData& A, const MeshData& B);

// Extrusion along path
MeshData extrudeMesh(const MeshData& profile, const std::vector<glm::vec3>& path);
```

Physics fields (MUGE compressed, resonance) are mapped to terrain height via: $y_{\text{terrain}} = \log_{10}(|g_{\text{MUGE}}|) \cdot \text{scale}$

6. MicroTeX LaTeX Overlay

```
// Render physics equations as LaTeX overlay via MicroTeX
void renderLatexOverlay(const std::string& latex_str,
float x, float y,
float font_size = 16.0f) {
auto tex_renderer = MicroTeX::create();
tex_renderer->render(latex_str, x, y, font_size);
}

// Example overlays:
renderLatexOverlay("$g_{\text{comp}} = 1.78 \times 10^{39}$", 0.1, 0.9);
renderLatexOverlay("$g_{\text{res}} = 1.66 \times 10^{45}$", 0.1, 0.85);
renderLatexOverlay("$F_U = -2.06 \times 10^{59}$", 0.1, 0.80);
```

7. UQFF Integration with Rendering

The simulation loop:

- Physics compute: `compute_compressed_MUGE()` / `compute_resonance_MUGE()`
- Field normalization: $g \propto \log_{10}(|g|)$ for visual range compression
- Terrain update: heightmap updated from new MUGE values each frame
- Particle update: `addUQFFBodyForce()` applied to fluid particles
- LaTeX HUD: Current MUGE values rendered as math equations in overlay
- Entity update: position + orientation updated from time-evolved UQFF

8. CP/Architecture Integration

This framework resides in **Tier 3 VR/VM** (`source2(HEAD PROGRAM).cpp`, ~2,625 lines, GPU-heavy).
Connection to CP calculators:

- CP1/CP2 compute MUGE values $\rightarrow$ stored in `uqff_results.json`
- `populate_simulation_entities()` reads `uqff_results.json` $\rightarrow$ creates 3D entities
- Source2 GUI (Tab for 3D Viewer) $\rightarrow$ renders entities via OpenGL pipeline
- Per-system plugin DLLs can call CP2/CP3 calculators at runtime

Status: $\rightarrow$ Complete | **CP Stage:** Architectural/Tier 3 VR/VM **Supersedes:** N/A (new framework) | **Related:** PAPER072 (*source2 GUI arch*), PAPER157 (Solar System entity params), PAPER168 connects to CP2/CP3 via `uqffresults.json`
